Async and Sync in Node.js — Complete Deep Dive

Start From Absolute Zero — What Does Your Computer Actually Do

Before touching a single line of code, you need to understand something fundamental about how computers work.

Your computer has a **CPU** (processor). The CPU is the brain — it actually executes your code, does calculations, runs logic. The CPU is extremely fast. It can perform billions of operations per second.

But your computer also has other components that the CPU needs to communicate with:

- **Hard disk / SSD** — where files are stored
- **RAM** — temporary memory
- **Network card** — for internet communication
- **Database** — which runs either locally or over a network

Here is the critical thing to understand:

The CPU is incredibly fast. Everything else is incredibly slow by comparison.

How slow? Let us put it in human terms. If one CPU operation takes 1 second in human time:

CPU operation → 1 second
RAM access → 6 minutes
SSD read → 4 days
Network request → 1 to 3 years

This is not an exaggeration. The speed difference between CPU operations and I/O operations is genuinely this dramatic.

This speed difference is **the entire reason async and sync exist.** Everything else follows from this one fact.

What is I/O

I/O stands for **Input/Output**. It refers to any operation where your program communicates with something outside the CPU.

I/O operations include:

- Reading a file from disk
- Writing a file to disk
- Querying a database
- Making an HTTP request to another server
- Reading from network
- Writing to network
- Accessing RAM in large chunks

All of these are dramatically slower than pure CPU computation. And all of these are at the heart of what Node.js applications do constantly.

A web server does almost nothing but I/O:

- Receive a request (network I/O)
- Query a database (network I/O)
- Read a file (disk I/O)
- Send a response (network I/O)

This is why the sync vs async distinction matters SO much in Node.js specifically.

Synchronous — The Simple but Problematic Way

What Synchronous Means

Synchronous means **one thing at a time, in order, waiting for each thing to finish before starting the next.**

Your code executes line by line. Line 1 finishes completely before line 2 starts. Line 2 finishes completely before line 3 starts.

```
console.log("Line 1")
console.log("Line 2")
console.log("Line 3")
```

Output:

```
Line 1
Line 2
Line 3
```

Simple and predictable. Exactly the order you wrote them.

Synchronous With I/O — The Problem

Now watch what happens when synchronous code hits an I/O operation:

```
import fs from 'node:fs'

console.log("Before reading file")

const data = fs.readFileSync('bigfile.txt')  // synchronous file read

console.log("After reading file")
console.log("Doing other work")
```

What happens here step by step:

1. "Before reading file" → prints immediately

2. fs.readFileSync('bigfile.txt') →
   CPU sends request to disk
   CPU now sits completely IDLE waiting
   Disk spins, finds the file, reads it
   This takes time...
   More time...
   Still waiting...
   Finally done
   Data returned to CPU

3. "After reading file" → prints

4. "Doing other work" → prints

During step 2, your **entire program is frozen.** Nothing else can happen. No other requests can be handled. No other code can run. The CPU is just sitting there, idle, waiting for the disk.

This is called **blocking** — the I/O operation blocks everything else.

Visualising Blocking

Imagine a single cashier at a supermarket (this is your single thread in Node.js).

Customer A comes to the cashier. The cashier scans the items. Then Customer A says "wait, I need to call my bank to check my balance."

In synchronous mode, the cashier **stands there and waits.** Does nothing. Talks to nobody. Just waits for the phone call to finish.

Meanwhile, 50 customers are queuing behind Customer A. Nobody is being served. The entire line is blocked.

This is synchronous I/O in a server application.

Synchronous Example in Node.js

```
import fs from 'node:fs'

// This is synchronous - notice the "Sync" at the end
const data = fs.readFileSync('./data.txt', 'utf8')
console.log(data)

// This line cannot run until readFileSync completely finishes
console.log("This waits for the file read to complete")
```

The `Sync` suffix in Node.js method names is your signal that this method is blocking. `readFileSync`, `writeFileSync`, `appendFileSync` — all blocking, all synchronous.

Asynchronous — The Node.js Way

What Asynchronous Means

Asynchronous means **start the work, but do not wait for it to finish. Move on immediately. When it finishes, come back and handle the result.**

Instead of blocking and waiting, you say: "Hey, go do this I/O work. When you are done, let me know and I will handle the result. In the meantime I will keep doing other things."

The Same Cashier — Async Version

Customer A comes to the cashier. Needs to call the bank. In async mode, the cashier says "No problem, step aside and make your call. I will serve the next customers. When your call finishes, come back to the front and I will finish your transaction."

Now Customer B gets served. Customer C gets served. Customer D gets served. Customer A's call finishes, they come back, cashier finishes their transaction.

The cashier (CPU/thread) is never idle. Always doing useful work.

How Node.js Makes Async Work — The Event Loop

The Single Thread Reality

Node.js runs on a **single thread.** This means there is literally one execution path. One thing runs at a time on the JavaScript side.

When people first hear this they think — "but how can Node.js handle thousands of requests simultaneously if it only has one thread?"

The answer is the **Event Loop.**

What the Event Loop Is

The Event Loop is the mechanism that makes async programming possible in Node.js. It is a continuously running loop that:

1. Checks if there is any JavaScript code to run
2. If yes, runs it
3. If no, checks if any async operations have completed
4. If yes, runs the callbacks for those completed operations
5. Goes back to step 1

It loops forever, constantly checking — "is there something to do? is there something to do? is there something to do?"

How Async I/O Actually Works

When you make an async I/O call in Node.js, here is what happens:

1. Your JavaScript code calls an async function
   (like fs.readFile or a database query)

2. Node.js (via libuv) hands this work to the

OPERATING SYSTEM

3. Your JavaScript thread is immediately FREE
   It does not wait. It moves to the next line.

4. The OS does the I/O work in the background
   (reading the file, querying the database, etc.)

5. When the OS finishes, it notifies Node.js

6. Node.js puts your callback function into
   a QUEUE called the callback queue

7. The Event Loop sees this callback in the queue

8. When the JavaScript thread is free (not running
   other code), the Event Loop picks up the callback
   and runs it

Your JavaScript thread is never blocked. It is always free to handle other things while I/O happens in the background.

The Three Ways to Write Async Code in Node.js

The transcript covers all three. Let us go through each one deeply.

Way 1 — Callbacks (The Original Way)

What a Callback Is

A callback is simply **a function you pass to another function, to be called later when some work is done.**

You are saying: "Here is a function. When you finish your work, call this function."

Basic Callback Example

```
import fs from 'node:fs'
```

```
console.log("Before reading")

fs.readFile('./data.txt', 'utf8', function(error, data) {
   // This function is the CALLBACK
   // It runs AFTER the file has been read
   console.log("File content:", data)
})

console.log("After calling readFile")
console.log("This runs immediately, not waiting for file")
```

Output:

```
Before reading
After calling readFile
This runs immediately, not waiting for file
File content: (whatever is in data.txt)
```

Notice the order. "After calling readFile" and "This runs immediately" print BEFORE "File content" — even though they are written after the `readFile` call in the code.

This is asynchronous behaviour. The file reading happens in the background. Your code keeps running. When the file is ready, the callback runs.

The Error-First Callback Pattern

This is a fundamental Node.js convention. Every callback in Node.js follows this pattern:

```
function myCallback(error, data) {
   // FIRST parameter is ALWAYS the error
   // SECOND parameter is the actual data
}
```

Why error first? Because it forces you to handle errors. You cannot accidentally ignore them.

```
fs.readFile('./data.txt', 'utf8', function(error, data) {
   // Always check error first
   if (error) {
      console.log("Something went wrong:", error.message)
      return  // stop here, do not try to use data
   }
```

```
    // If we reach here, no error occurred
    console.log("File content:", data)
})
```

The `return` after handling the error is important. Without it, your code would continue to the `console.log(data)` line even when there was an error and `data` is undefined.

A Real Example — Writing Then Appending

This is exactly what the transcript showed. What if you need to write a file and then append to it?

```
import fs from 'node:fs'

// Write the file first
fs.writeFile('./hello.txt', 'Hello Node.js\n', function(error) {
    if (error) {
        console.log("Write failed:", error)
        return
    }

    console.log("File written successfully")

    // NOW append - inside the callback of writeFile
    fs.appendFile('./hello.txt', 'Hello JavaScript\n', function(error) {
        if (error) {
            console.log("Append failed:", error)
            return
        }

        console.log("File appended successfully")
    })
})
```

Why is the appendFile INSIDE the writeFile callback? Because of the race condition the transcript explained.

The Race Condition — Why Order Matters

Look at this broken code:

```
// BROKEN - race condition
fs.writeFile('./hello.txt', 'Hello Node.js\n', function(error) {
```

```
    console.log("Written")
})

fs.appendFile('./hello.txt', 'Hello JavaScript\n', function(error) {
    console.log("Appended")
})
```

Both operations start at almost the same time. Both go to the OS asynchronously. The OS might do them in any order. The append might happen before the write finishes, corrupting the file.

This is a **race condition** — two operations racing to access the same resource, with unpredictable results.

The fix is nesting — only start the append inside the write's callback, guaranteeing the write finishes first.

Callback Hell — The Problem With Nesting

What if you need many sequential operations?

```
fs.readFile('./config.txt', 'utf8', function(error, config) {
    if (error) return console.log(error)

    fs.readFile('./users.txt', 'utf8', function(error, users) {
        if (error) return console.log(error)

        fs.writeFile('./output.txt', config + users, function(error) {
            if (error) return console.log(error)

            fs.appendFile('./log.txt', 'Done\n', function(error) {
                if (error) return console.log(error)

                fs.readFile('./output.txt', 'utf8', function(error, result) {
                    if (error) return console.log(error)

                    console.log("Final result:", result)
                    // Imagine needing even more operations here...
                })
            })
        })
    })
})
```

This is **Callback Hell** — also called the **Pyramid of Doom** because of its triangular shape. The code keeps indenting further and further to the right.

Problems with callback hell:

- Nearly impossible to read
- Very hard to debug
- Error handling is duplicated everywhere
- Adding new steps requires restructuring everything
- Maintaining this code is a nightmare

This is exactly why Promises were created.

Way 2 — Promises (The Better Way)

What a Promise Is

A Promise is an object that represents **the eventual result of an async operation.** It is a placeholder for a value that does not exist yet but will exist in the future.

A Promise is always in one of three states:

```
PENDING   → the async operation is still running
FULFILLED → the async operation completed successfully, has a value
REJECTED  → the async operation failed, has an error
```

Once a Promise moves from PENDING to either FULFILLED or REJECTED, it never changes state again.

How to Use Promises

To use the Promise-based version of the fs module:

```
import fs from 'node:fs/promises'
// Notice the /promises at the end
// This gives us the Promise version of all fs methods
```

Now `fs.readFile` returns a Promise instead of using callbacks:

```
import fs from 'node:fs/promises'
```

```
console.log("Before reading")

const promise = fs.readFile('./data.txt', 'utf8')
// promise is now a Promise object in PENDING state
// The file is being read in the background

console.log("After calling readFile - file not ready yet")

promise.then(function(data) {
   // This runs when the Promise is FULFILLED (file read successfully)
   console.log("File content:", data)
})

promise.catch(function(error) {
   // This runs when the Promise is REJECTED (something went wrong)
   console.log("Error:", error)
})
```

Chaining Promises — Solving Callback Hell

The real power of Promises is chaining. Instead of nesting, you chain with `.then()`:

```
import fs from 'node:fs/promises'

fs.writeFile('./hello.txt', 'Hello Node.js\n')
   .then(function() {
      console.log("Written successfully")
      return fs.appendFile('./hello.txt', 'Hello JavaScript\n')
   })
   .then(function() {
      console.log("Appended successfully")
      return fs.readFile('./hello.txt', 'utf8')
   })
   .then(function(data) {
      console.log("Final file content:", data)
   })
   .catch(function(error) {
      // ONE catch handles errors from ANY step above
      console.log("Something went wrong:", error)
   })
```

Compare this to callback hell. The same operations, but:

- Flat structure, no deep nesting
- One `.catch()` handles all errors
- Easy to read top to bottom
- Easy to add or remove steps

How .then() Works

`.then()` takes a function. When the Promise fulfills, it calls that function with the result.

If your `.then()` function **returns another Promise**, the chain waits for that Promise to fulfill before calling the next `.then()`.

This is how you sequence async operations without nesting — each `.then()` waits for the previous one to complete.

Way 3 — Async/Await (The Modern Way)

What Async/Await Is

Async/Await is **syntax built on top of Promises.** It does not replace Promises — it uses them. But it lets you write async code that LOOKS synchronous.

Instead of `.then()` chains, you write code that reads top to bottom like normal synchronous code — but runs asynchronously underneath.

The Two Keywords

`async` — placed before a function declaration. Means "this function contains async operations and will return a Promise."

`await` — placed before a Promise. Means "pause execution of THIS function here until this Promise resolves, then give me the value."

Basic Example

```
import fs from 'node:fs/promises'

async function readMyFile() {
  console.log("Starting to read file")

  const data = await fs.readFile('./data.txt', 'utf8')
  // Execution PAUSES here until the file is read
```

```
    // data now contains the file contents

    console.log("File content:", data)
    console.log("This runs AFTER the file is read")
}

readMyFile()
console.log("This runs WHILE the file is being read")
```

Output:

```
Starting to read file
This runs WHILE the file is being read
File content: (file contents here)
This runs AFTER the file is read
```

The Critical Distinction — Suspend vs Block

This confused many students in the transcript. The instructor addressed it directly.

When you use `await`, the **function is suspended** at that line. But the **thread is NOT blocked.**

```
Synchronous (readFileSync):
  Thread is BLOCKED
  Cannot do anything else
  Other requests queue up
  Performance degrades
  BAD for servers

Async/Await (await readFile):
  Function is SUSPENDED at the await line
  Thread is FREE to do other things
  Other requests get handled
  Performance stays good
  GOOD for servers
```

Think of it this way:

Blocking (sync): You are cooking pasta. You stand at the stove staring at the water, doing nothing else, waiting for it to boil.

Suspended (async/await): You are cooking pasta. You put the water on, then go do other things — clean the kitchen, prepare the sauce, respond to messages. When the water boils, you come back and add the pasta.

You personally (the thread) are not standing there wasting time. The water (I/O) is heating up on its own. You come back when it signals it is ready.

Sequential Operations With Async/Await

The same write-then-append example, using async/await:

```
import fs from 'node:fs/promises'

async function processFile() {
    try {
        await fs.writeFile('./hello.txt', 'Hello Node.js\n')
        console.log("Written successfully")

        await fs.appendFile('./hello.txt', 'Hello JavaScript\n')
        console.log("Appended successfully")

        const data = await fs.readFile('./hello.txt', 'utf8')
        console.log("File content:", data)

    } catch (error) {
        console.log("Something went wrong:", error)
    }
}

processFile()
```

Look at how clean this is. It reads exactly like synchronous code — top to bottom, one operation after another. But it is completely asynchronous. The thread is never blocked.

Error Handling With Try/Catch

With async/await, you use the familiar `try/catch` pattern for error handling:

```
async function readFile() {
    try {
        const data = await fs.readFile('./data.txt', 'utf8')
        console.log(data)
    } catch (error) {
        // Any error from the await above lands here
```

```
        console.log("Failed to read file:", error.message)
    }
}
```

This is much cleaner than callback's error-first pattern or Promise's `.catch()` chains.

The Proof — Event Loop in Action

The transcript showed a specific example to prove async behaviour. Let me recreate it completely:

```
console.log("Learn")

setTimeout(() => {
    console.log("Build")
}, 0)   // zero milliseconds delay

console.log("Success")
```

What do you think the output is?

Most beginners say:

```
Learn
Build
Success
```

The actual output is:

```
Learn
Success
Build
```

Why?

Even though the delay is 0 milliseconds, `setTimeout` is asynchronous. Here is exactly what happens:

1. "Learn" → prints immediately (synchronous)

2. setTimeout encountered →
   The callback function is handed to the Event Loop
   It is scheduled to run after 0ms
   JavaScript does NOT wait
   Moves immediately to next line

3. "Success" → prints immediately (synchronous)

4. All synchronous code is now done
   Thread is free

5. Event Loop checks the queue →
   setTimeout callback is there and ready

6. "Build" → prints

The `setTimeout` callback goes to the callback queue. The Event Loop only runs callbacks from the queue when the main thread is completely free — after all synchronous code has finished.

Even with 0 delay, async operations always run after synchronous code completes.

Sync vs Async — When to Use Each

Use Synchronous When:

Application startup — reading config files before your server starts is fine synchronously because no requests are being handled yet:

```
// Fine at startup - no requests being handled yet
const config = fs.readFileSync('./config.json', 'utf8')
const settings = JSON.parse(config)

// Now start the server with these settings
startServer(settings)
```

Command line scripts — simple scripts you run manually where there is only one user (you):

```
// A script to process a file - only one user, no concurrency
```

```
const data = fs.readFileSync('./input.txt', 'utf8')
const processed = processData(data)
fs.writeFileSync('./output.txt', processed)
```

Simple utilities — tools that run once and exit, not servers:

```
// Quick utility script
const files = fs.readdirSync('./')
console.log("Files in current directory:", files)
```

Use Asynchronous When:

Web servers — always. A server handles many simultaneous requests:

```
// Server handler - MUST be async
app.get('/users', async (req, res) => {
   const users = await database.findAll()
   res.json(users)
})
```

Any application with concurrent users — if multiple people can use your app at the same time, blocking one user blocks everyone:

Long I/O operations — reading large files, making multiple network requests, complex database queries.

The Performance Difference — Why It Matters

Imagine 1000 users hit your server simultaneously. Each request needs to read a file that takes 100ms.

Synchronous server:

```
User 1:    0ms → 100ms   (reading file, BLOCKING)
User 2:    100ms → 200ms (waiting for user 1, then reading)
User 3:    200ms → 300ms (waiting for users 1 and 2)
...
User 1000: 99,900ms → 100,000ms (waits for all 999 others)

User 1000 waits 100 SECONDS for a response
```

Asynchronous server:

All 1000 users: 0ms → ~100ms (all files read simultaneously)

Every user gets their response in roughly 100ms

This is not an exaggeration. This is the real difference between a server that can handle production traffic and one that falls over under load.

Complete Comparison Table

Aspect	Synchronous	Callbacks	Promises	Async/Await
Blocks thread	YES	No	No	No
Readability	Simple	Gets messy	Better	Best
Error handling	try/catch	Error first param	.catch()	try/catch
Nesting problem	No	Yes (callback hell)	No (chains)	No
Use in production	Only startup	Older code	Good	Recommended
Node.js suffix	`Sync`	No suffix	`/promises` import	`/promises` import

The Golden Rule

The transcript stated this clearly and it is the most important takeaway:

Never use synchronous I/O in a server that handles multiple users.

The moment you use `readFileSync`, `writeFileSync`, or any `*Sync` method inside a request handler, you are blocking your entire server for every user while that operation runs.

Always use async/await with the Promise-based APIs for any I/O in production server code. Your server stays responsive, handles thousands of concurrent users, and never makes one user wait because of another user's request.

This is not just best practice — it is the entire reason Node.js was built the way it was built. Using synchronous I/O in Node.js defeats the entire purpose of choosing Node.js in the first place.